Experiment No. – 4						
Date of Performance:	**05/02/2026**					
Date of Submission:	**12/02/2026**					
Conceptual Understanding (Cognitive) (20%) (02)	Code Execution (Psychomotor) (15%) (1.5)	Problem-Solving & Debugging (Cognitive + Psychomotor) (20%) (02)	Professional Conduct & Lab Ethics (Affective) (20%) (02)	Output / Result Accuracy (Product) (15%) (1.5)	Documentation & Result Analysis (Cognitive) (10%) (01)	Sign with Date

Experiment No. 4

1.1 Aim: Develop, deploy and execute smart contract using Metamask, Remix and web3.

1.2 Course Outcome: Develop the application using the concept of Ethereum with smart contracts using different development tools.

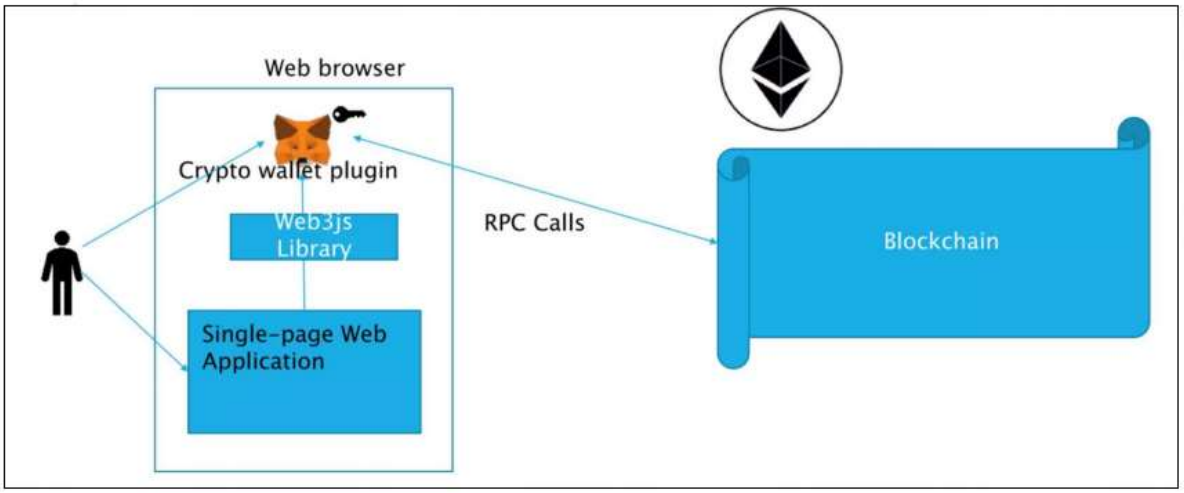
1.3 Learning Objectives: Develop smart contract using Metamask, Remix and web3.

1.4 Requirement: MetaMask, Remix IDE, Web3.js, Node.js, React.js, Solidity

1.5 Related Theory:

Metamask:

Metamask is a cryptocurrency wallet and browser extension that allows users to interact with decentralized applications (dapps) on the Ethereum blockchain. It functions as a bridge between a user's web browser and the Ethereum blockchain, allowing them to manage their Ethereum accounts, send and receive Ethereum and other ERC-20 tokens, and access decentralized applications directly from their browser.
Metamask is a popular tool for Ethereum users who want to securely store their digital assets and participate in the growing ecosystem of decentralized applications, such as decentralized finance (DeFi) platforms, non-fungible token (NFT) marketplaces, and decentralized exchanges (DEXs). It is available as a browser extension for Google Chrome, Firefox, and Brave, as well as a mobile app for iOS and Android.



WEB3.JS:
Web3.js is a library that allows developers to interact with decentralized applications (dApps) built on the Ethereum blockchain using JavaScript. Some of the ways that web3.js can be used
to create dApps include:Sending transactions: web3.js allows developers to create, sign, and send transactions on the Ethereum blockchain.
WEB3.JS CODE:

```
export const loadWeb3 = async () => {
if (window.ethereum) {
window.web3 = new Web3(window.ethereum);
await window.ethereum.enable();
} else if (window.web3) {
window.web3 = new Web3(window.web3.currentProvider);
} else {
window.alert(
"Non-Ethereum browser detected. You should consider trying MetaMask!"
);
}
};
export const loadBlockchainData = async () => {
const web3 = window.web3;
// Load account
const accounts = await web3.eth.getAccounts();
// Network ID
```

```
const networkId = await web3.eth.net.getId();
// Network data
if (networkId) {
const auth = new web3.eth.Contract(
Demo.abi,
Demo.networks[networkId].address
);
return { auth, accounts: accounts[0] };
}
}
```

1.6 Procedure:

Step 1: Go to the client in the folder and open cmd.
Step 2: Type npm start.
Step 3: Frontend end will pop up where the Metamask is installed. If there is no metamask connected to the web browser, it will give an error dialogue box.
Step 4: Once the frontend of opens up, add details according to the requirements. AKASH MAKWANA BE6 ROLLNO:23
Step 5: Then submit, the metamask dialogue box will appear and you will have all the details
about the account.
Step 6: A confirmation of the payment will be asked and click confirm button.
Step 7: Once the payment is confirmed, your details will be displayed.

1.7 Program and Output:

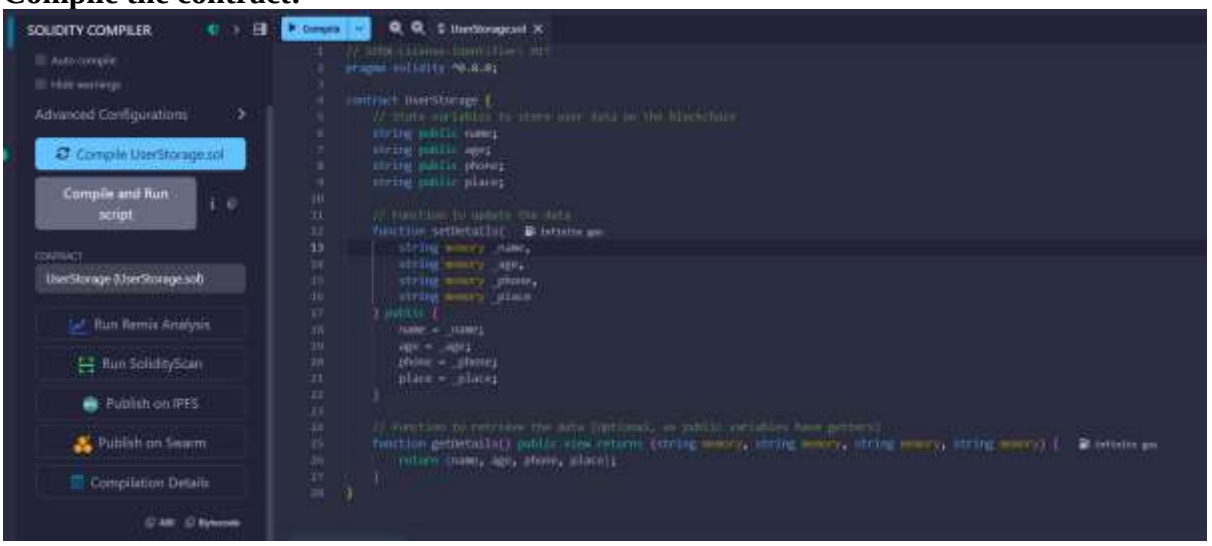
1) **Deploy the contract on Metamask through Remix IDE**

Code:-

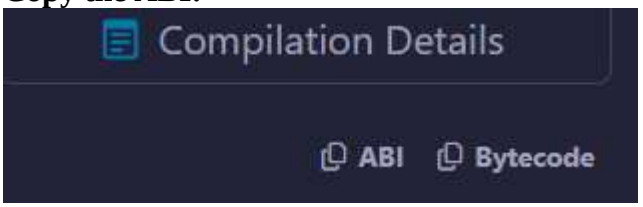
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract UserStorage {
   // State variables to store user data on the blockchain
   string public name;
   string public age;
   string public phone;
   string public place;

   // Function to update the data
   function setDetails(
      string memory _name,
      string memory _age,
      string memory _phone,
```

```
        string memory _place
    ) public {
        name = _name;
        age = _age;
        phone = _phone;
        place = _place;
    }

    // Function to retrieve the data (Optional, as public variables have getters)
    function getDetails() public view returns (string memory, string memory, string memory,
string memory) {
        return (name, age, phone, place);
    }
}
```

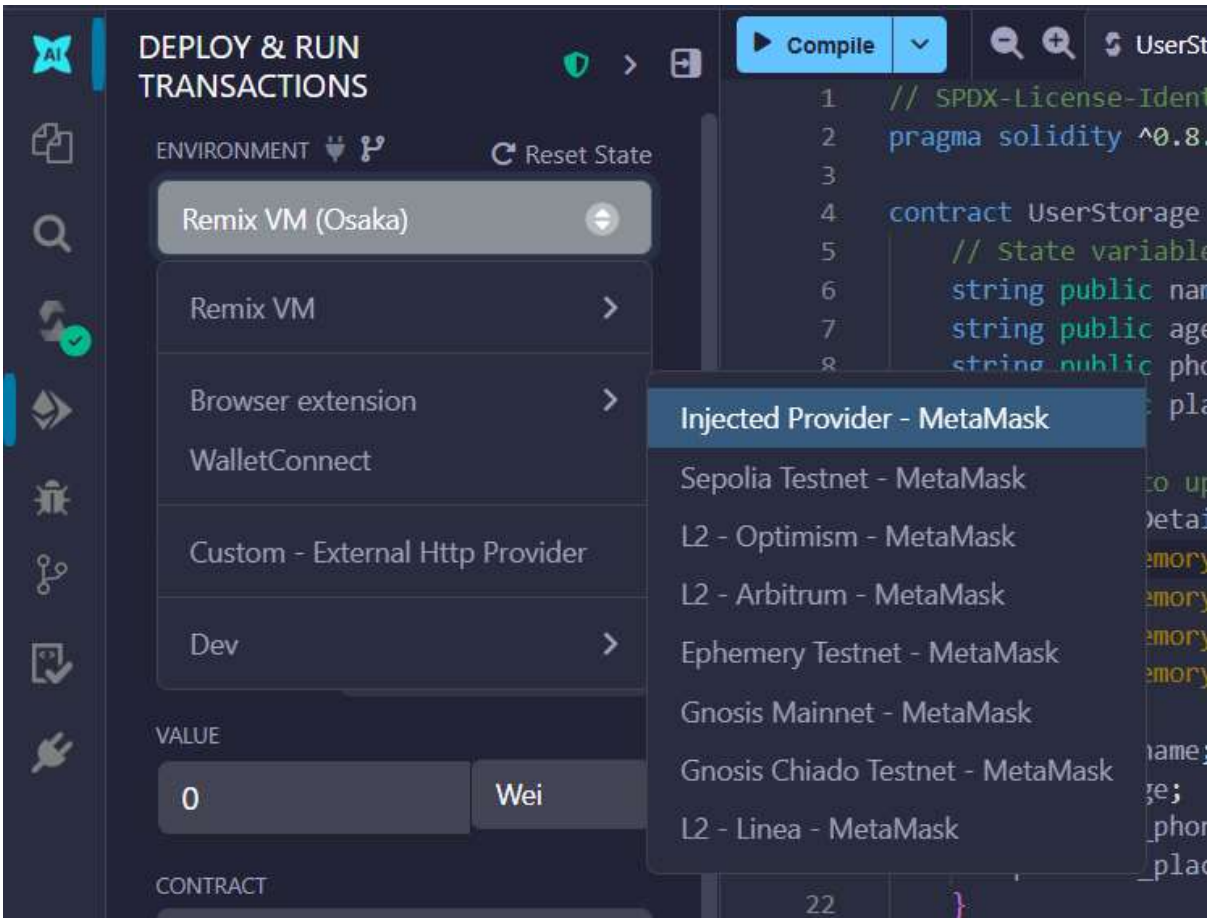
Compile the contract:-



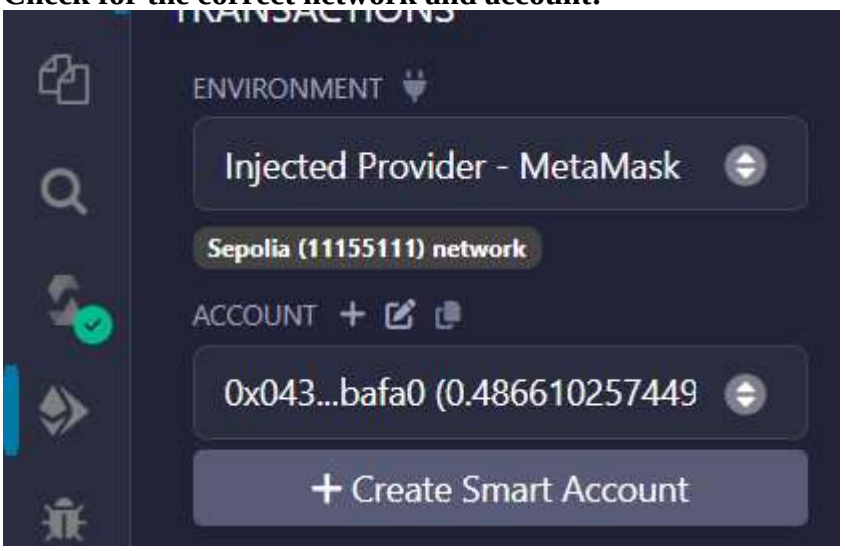
Copy the ABI:-



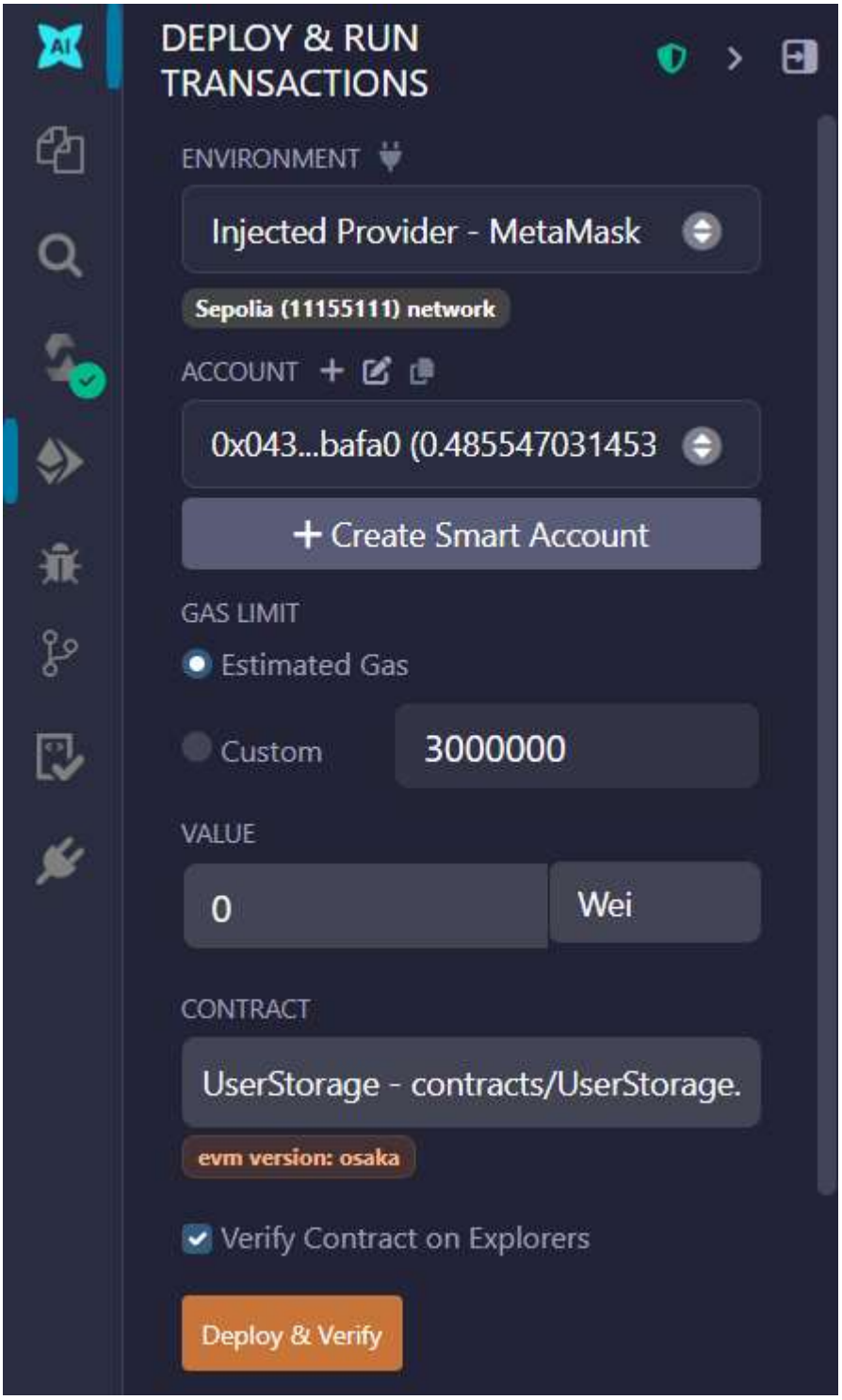
Select the Metamask environment:-



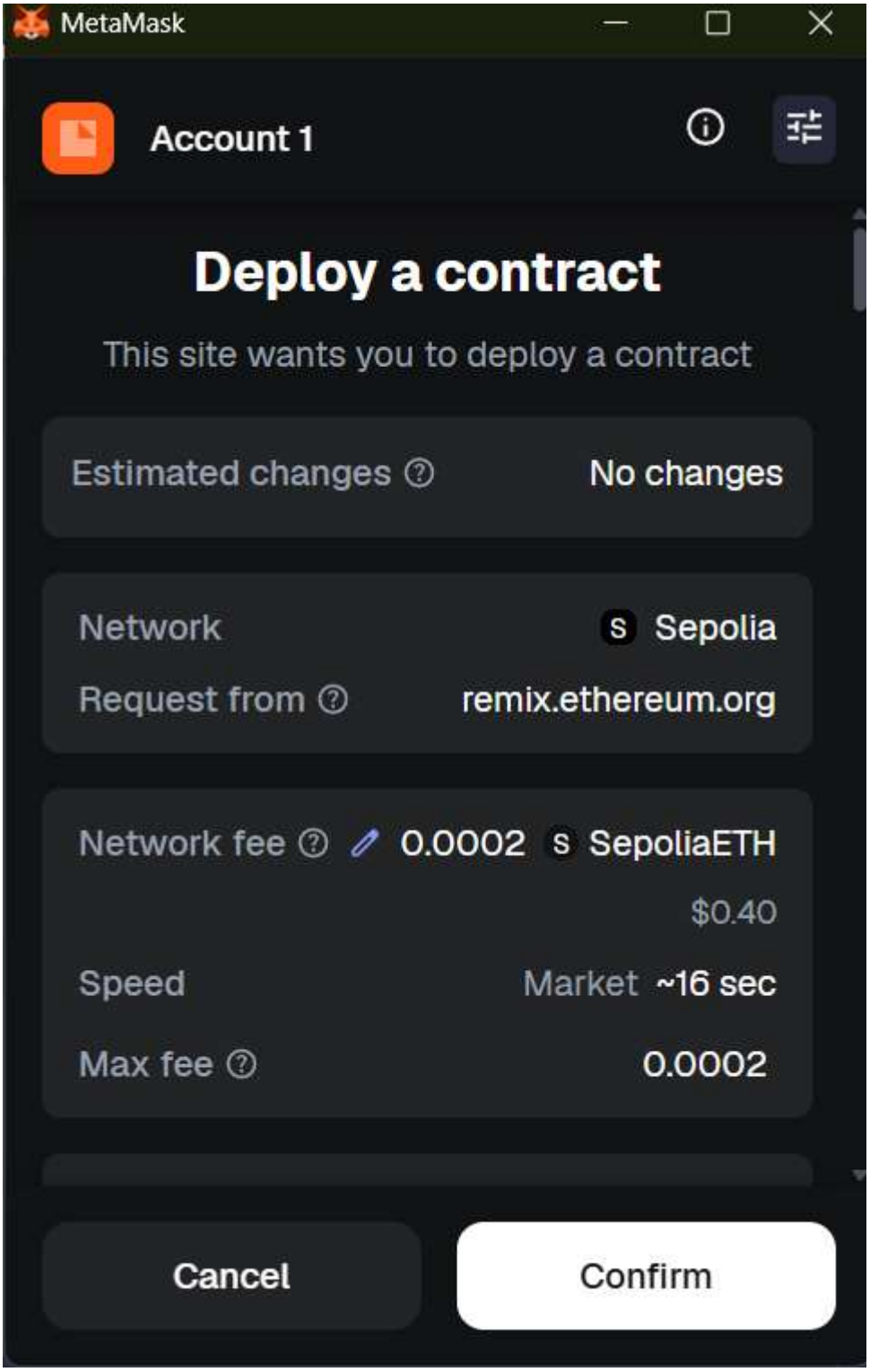
Check for the correct network and account:-



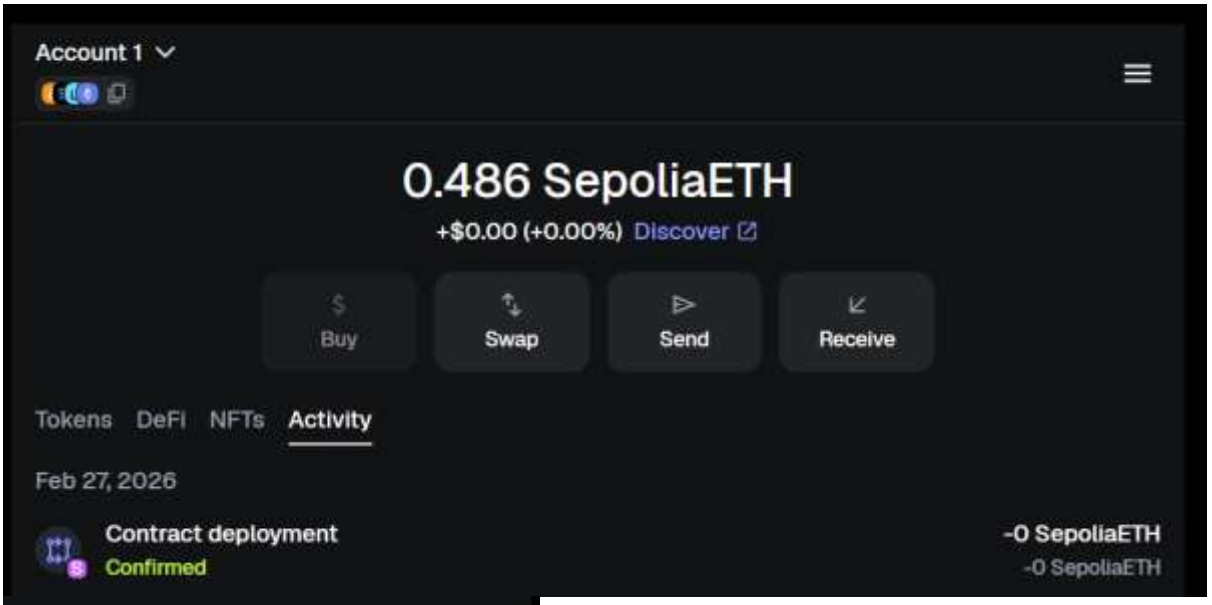
Click the "Deploy & Verify" button:-

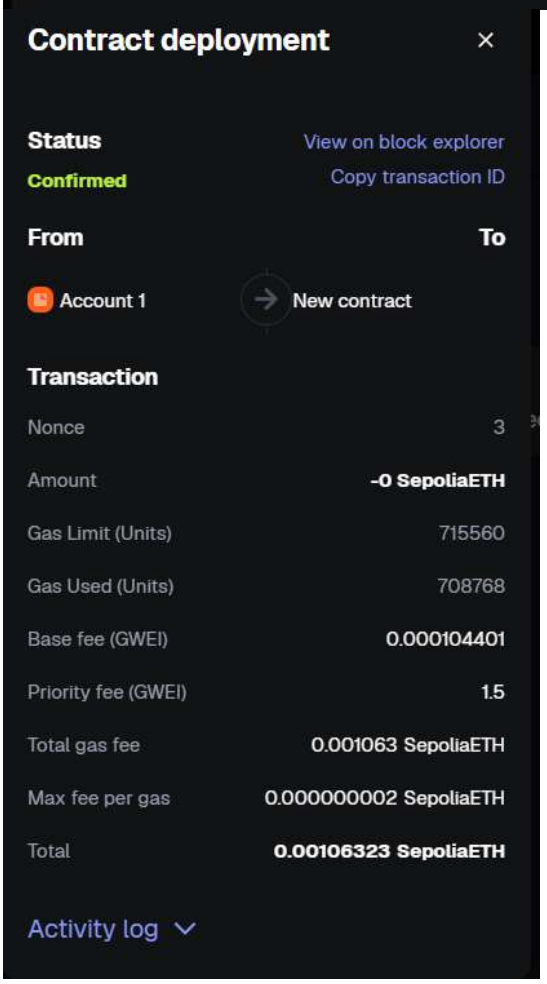


Click the "Confirm" button:-

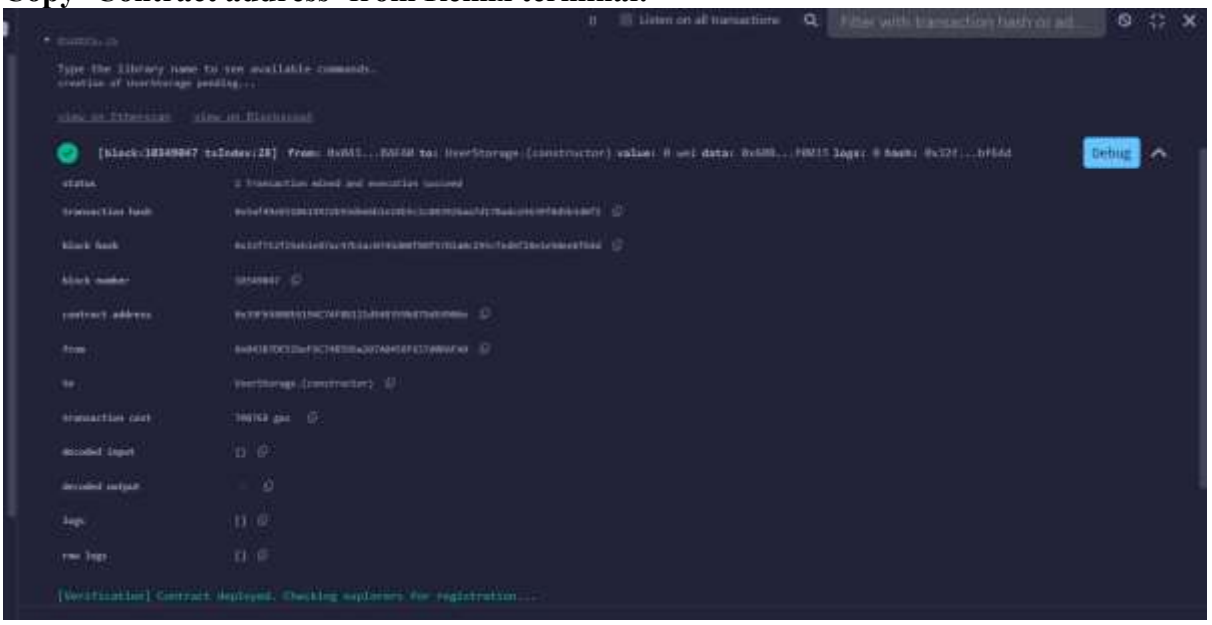


See the recorded transaction on Metamask network:-





Copy ‘Contract address’ from Remix terminal:-



2) **Create the React app**

CMD commands:-

- npx create-react-app <app name>
- cd <app name>
- npm install web3
- **For VS Code (optional):-**
    - code .

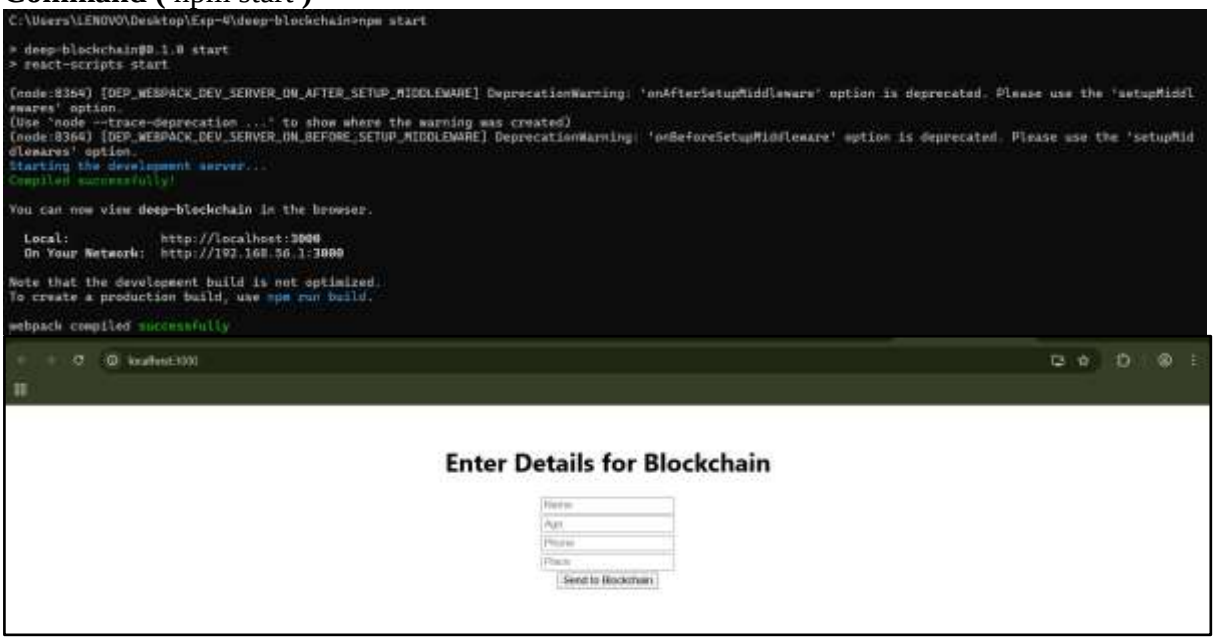
Code(App.js):-

```
import React, { useState } from 'react';
import Web3 from 'web3';

// PASTE YOUR CONTRACT ADDRESS FROM REMIX HERE
const CONTRACT_ADDRESS = "<Contract address>";

// PASTE YOUR ABI FROM REMIX HERE
const CONTRACT_ABI = <ABI>;

function App() {
  const [name, setName] = useState('');
  const [age, setAge] = useState('');
  const [phone, setPhone] = useState('');
  const [place, setPlace] = useState('');

  const submitData = async () => {
    if (window.ethereum) {
      const web3 = new Web3(window.ethereum);
      const accounts = await window.ethereum.request({ method: 'eth_requestAccounts' });
      const contract = new web3.eth.Contract(CONTRACT_ABI, CONTRACT_ADDRESS);
```

```
    await contract.methods.setDetails(name, age, phone, place).send({ from: accounts[0] });
    alert("Transaction Sent to Blockchain!");
  }
};

return (
  <div style={{ textAlign: 'center', marginTop: '50px' }}>
    <h1>Enter Details for Blockchain</h1>
    <input placeholder="Name" onChange={e => setName(e.target.value)} /><br/>
    <input placeholder="Age" onChange={e => setAge(e.target.value)} /><br/>
    <input placeholder="Phone" onChange={e => setPhone(e.target.value)} /><br/>
    <input placeholder="Place" onChange={e => setPlace(e.target.value)} /><br/>
    <button onClick={submitData}>Send to Blockchain</button>
  </div>
);
}

export default App;
```

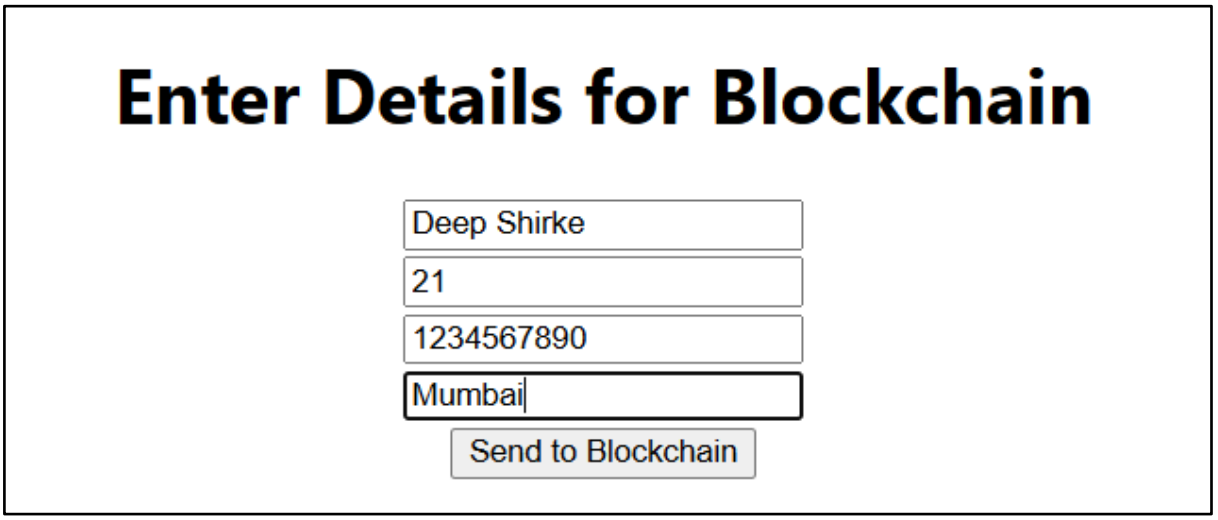
Start the server and access the app:-

Command (npm start **)**

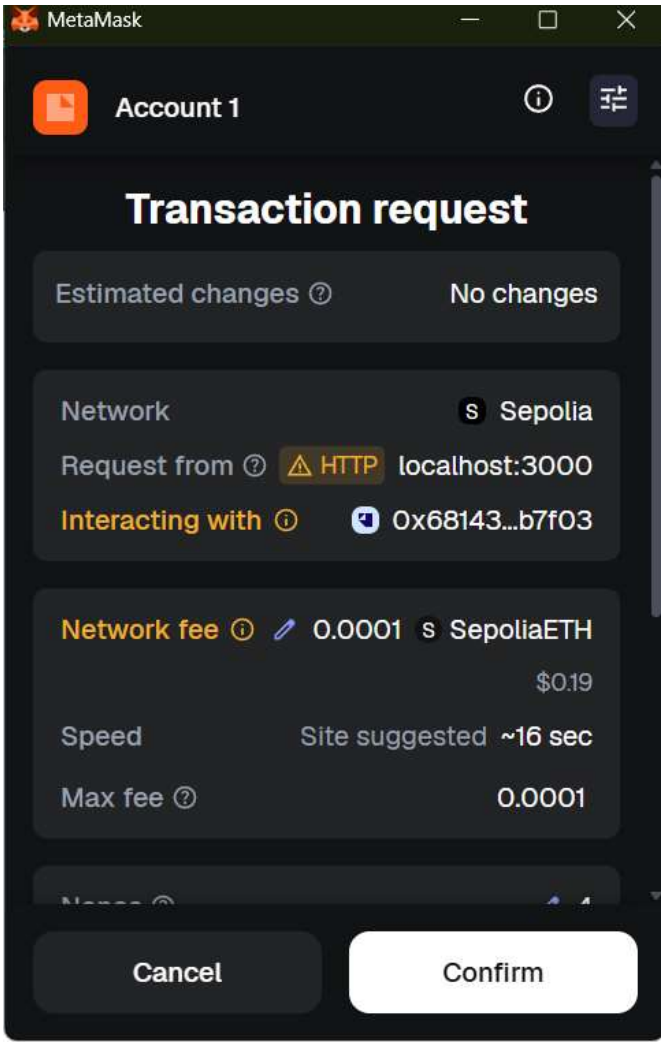


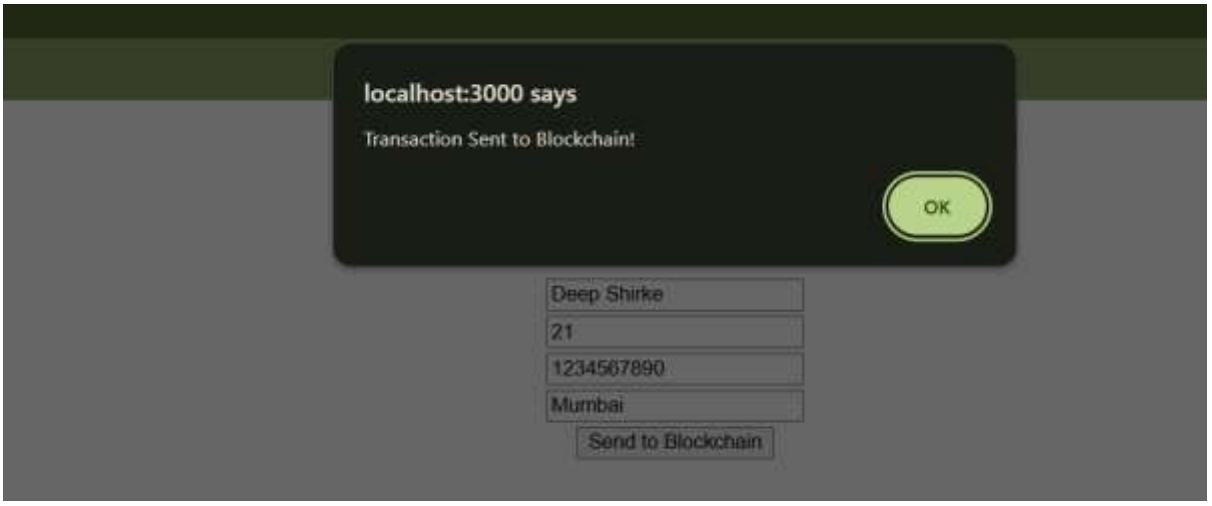
3) Carry out the transaction

Fill the details and click the "Send to Blockchain" button:-

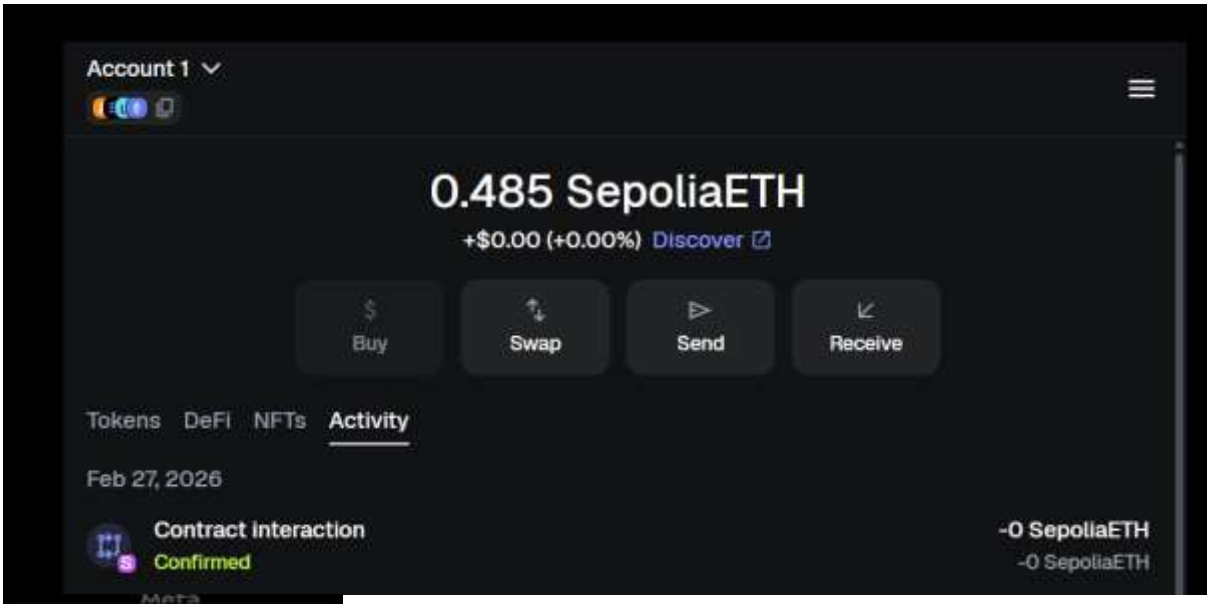


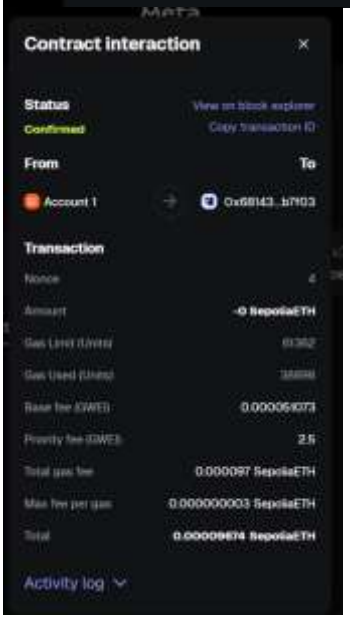
Click the "Confirm" button:-



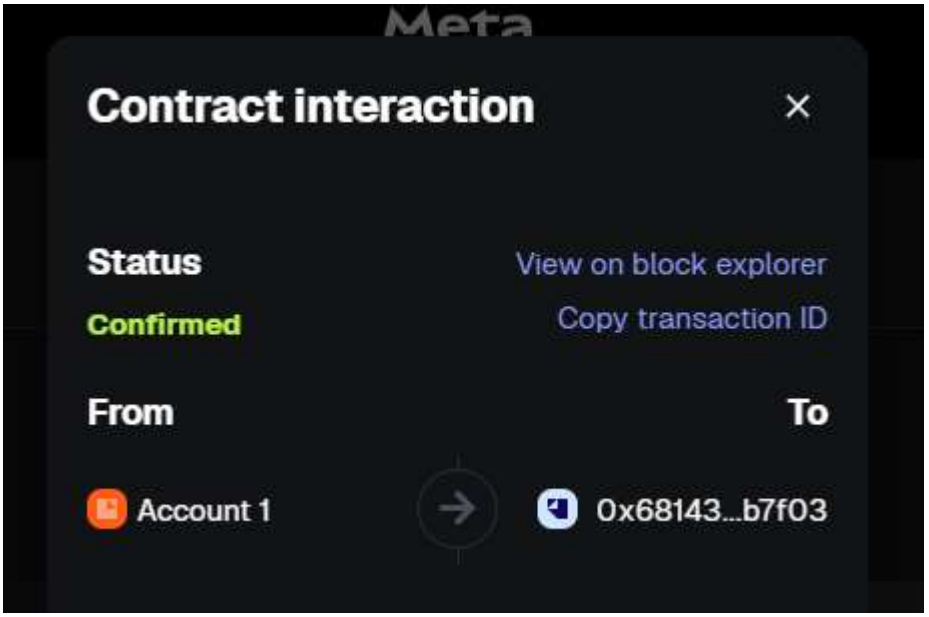


View the transaction on Metamask:-

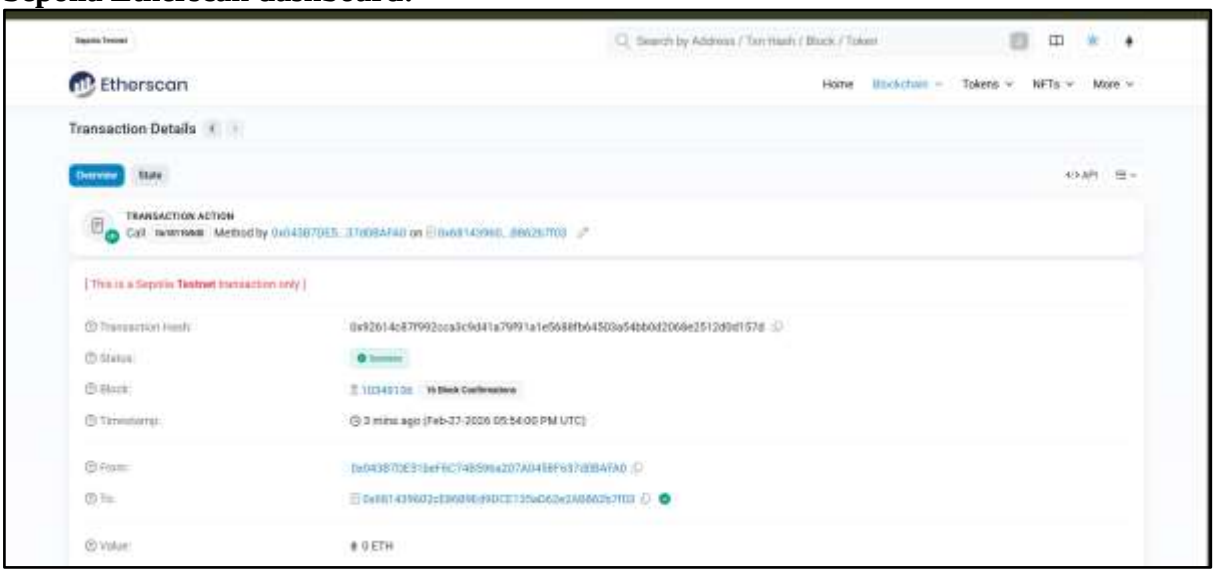




Click on the 'View on block explorer' link in Contract interaction:-



Sepolia Etherscan dashboard:-



Click on "Click to show more" option in 'More details':-

Click the "Decode Input Data" button:-

#	Name	Type	Data
0	_name	string	Deep Shirke
1	_contact	string	21
2	_residenceAddress	string	1234567890
3	_aadharNumber	string	Mumbai

Explore the other options if interested…….

<u>1.8 Conclusion</u>: In this Experiment we have learned about web3 also designed and deployed smart contract using Metamask and web3.